

In this article, we don't have code that needs to emit a signal, so you won't be seeing this in action.

Anyway, back to the code—the next thing we do in the constructor is fill up the list of fortunes with a bunch of `QByteArrays`. We don't use `QStrings` here because `QTcpSocket's` 'send' function works natively with `QByteArrays`, and `QByteArrays` can be constructed with standard C-strings; so this makes the code a lot easier. We won't be able to do fancy text-processing, but we don't need to. And that's it for the constructor.

The destructor comprises just the following three lines of actual code:

```
FortuneServer::~FortuneServer()
{
    // shut down the server first

    server->close();

    // disconnect all signals and slots connected to
    // this server

    server->disconnect();

    // and finally, queue this object for deletion at
    // the first opportune moment

    server->deleteLater();

    // and that's it
}
```

I won't even attempt to explain this one, except to mention that the `disconnect()` method of any `QObject`-derived class disconnects all the signals and slots connected to an object of that class.

We don't delete any `QObject`-derived object the standard C++ way because there might be pending signals that must be processed by the object. If we delete the object and Qt tries to deliver a signal to it, the program will segfault and make a mess of itself. We call the `deleteLater()` method (which is actually defined as a slot, and we'll use that property in the next function we define), and this makes sure there're no pending tasks for the object before pulling the plug on it.

Now for the one slot we have defined in the *class* -

sendFortune():

```
void FortuneServer::sendFortune()
{
    // we'll grab a client socket off of the server
    // first

    QTcpSocket * socket = server->nextPendingConnection();

    // now we'll wait until the socket is connected

    if (!(socket->waitForConnected())) {
        qDebug() << socket->errorString();
        return;
    }

    // now we'll choose a random fortune and send it to the
    // reciever

    socket->write(fortunes.at(qrand() % fortunes.size()));

    // we'll now tear down the connection

    socket->disconnectFromHost();
    connect(socket, SIGNAL(disconnected()), socket,
    SLOT(deleteLater()));

    // and we're done
}
```

This is also fairly self-explanatory. `QTcpServer::nextPendingConnection()` returns a pointer to a `QTcpSocket` (which is like a client socket, if you've done BSD socket programming in C). We then wait until the socket is connected, which we don't necessarily need to, since `nextPendingConnection()` is supposed to return a connected socket. But in Qt, a `QTcpSocket` doesn't emit a signal when it's ready for us to start writing, so let's wait a little.

`qDebug()` works just like `std::cout`, except that it automatically inserts a new line at the end of every statement; so I don't have to attach a `'\n'` or `std::endl` at the end of every line. Notice that `qDebug()` isn't a stream, but rather a stream

THE COMPLETE MAGAZINE ON OPEN SOURCE

OpenSource
ForYou

Your favourite Magazine on Open
Source is now on the Web, too.

OpenSourceForU.com

Follow us on Twitter@LinuxForYou